Master Engineering Systems

Major Project Report: Progress

Developing a Scalable Computer-Vision Pipeline for Human–Interaction Element Detection in Hospital Environments

Pavel Petrovski

HAN University of Applied Sciences
Supervisors: Siddharth Ajaykumar, Jan Benders, Ben Pyman

March 18, 2026

Master Engineering Systems – Major Project
Cyber-Physical Systems
HAN University of Applied Sciences
Arnhem

Preface

This report presents the work carried out as part of the Master thesis project in Cyber-Physical Systems at HAN University of Applied Sciences. The project was conducted in collaboration with Ambyon and HAN Automotive Research.

The objective of the thesis is to design and evaluate a scalable computer-vision pipeline for interaction-point perception in hospital environments. All work described in this report was performed by the author.

Disclaimer

This Project Report was written independently by the author as part of the Master of Engineering Systems program at HAN University of Applied Sciences.

During the preparation of this report, the generative artificial intelligence tool *ChatGPT* (OpenAI, 2025) was used selectively to support the writing process. The tool was employed to assist with improving sentence structure, refining academic language, and suggesting minor text rephrasing.

All technical content, system design decisions, experimental implementation, training procedures, evaluation methods, and results presented in this report are the original work of the author. The use of the AI tool did not replace independent analysis, interpretation of scientific literature, or critical reasoning.

The use of AI complies with the *MES Policy on the Use of Generative AI*, ensuring informed, transparent, ethical, and responsible application.

Summary

This project focuses on the design and evaluation of a computer-vision pipeline for detecting and interpreting interaction points, such as elevator buttons, in hospital environments. The work is motivated by the need for robust perception capabilities in service robots operating across different hospital infrastructures.

A scalable workflow is proposed that combines instance segmentation, semi-automatic annotation, and iterative model retraining. A custom dataset is created by extending existing data with instance-level annotations using a CVAT-based annotation pipeline. Multiple instance segmentation models are trained using transfer learning, evaluated using standard and task-oriented metrics, and prepared for deployment on embedded hardware.

The project demonstrates a complete perception pipeline, from dataset creation to embedded deployment, and provides a foundation for future extensions toward fully autonomous robot interaction in clinical environments.

Acronyms and Abbreviations

AI Artificial Intelligence.

AP Average Precision.

APA American Psychological Association.

CNN Convolutional Neural Network.

COCO Common Objects in Context.

CPU Central Processing Unit.

CV Computer Vision.

CVAT Computer Vision Annotation Tool.

DL Deep Learning.

FP16 Half-precision floating point format.

FPS Frames Per Second.

GPU Graphics Processing Unit.

IEEE Institute of Electrical and Electronics Engineers.

IoU Intersection over Union.

mAP Mean Average Precision.

Mask R-CNN Mask Region-Based Convolutional Neural Network.

ML Machine Learning.

NVIDIA Jetson Embedded GPU platform for edge AI.

OCR Optical Character Recognition.

ONNX Open Neural Network Exchange.

RGB Red–Green–Blue.

RGB-D Red–Green–Blue plus Depth.

RNN Recurrent Neural Network.

ROS Robot Operating System.

ROS 2 Robot Operating System 2.

SLAM Simultaneous Localization and Mapping.

SOTA State of the Art.

TBD To Be Determined.

TensorRT TensorRT.

WSL2 Windows Subsystem for Linux 2.

XML Extensible Markup Language.

YOLO You Only Look Once.

Contents

1 Introduction

This introduction includes the background information, problem definition, project objective and the research questions.

1.1 Background

Robotics in healthcare is moving from experimental pilots to practical deployment, supporting tasks such as logistics, delivery of medical supplies, and environmental assistance. These systems are intended not to replace clinical staff, but to support them by reducing repetitive physical work and enabling more time for patient care [6, 14]. As hospitals face increasing operational pressure and staff shortages, autonomous robots capable of safe navigation and interaction with the environment hold significant potential.

Hospitals are typically multi-floor environments that rely on infrastructure such as elevators, automatic doors, and access-control systems. For a robot to move freely and perform tasks across different floors, it must be able to detect and interact with these human–machine interfaces, including elevator buttons and door panels. Reliable perception of such interaction elements is therefore a key capability for achieving truly autonomous navigation in hospital settings. Without this, robots remain confined to limited single-floor operation or require human assistance for vertical transport.

Figure 1.1: CareBot prototype at HAN Automotive Research

At *HAN Automotive Research*, the CareBot platform is being developed as an educational and research project that allows students and researchers to experiment with robotic perception, planning, and control in realistic environments. While not designed to replicate the Ambyon ONE robot, the CareBot in this project will be equipped with a similar sensor stack and processing unit, enabling realistic simulation of Ambyon's operational conditions. The Ambyon ONE itself is a service robot currently under development by *Ambyon*, a Dutch startup focused on robotics for healthcare and logistics applications.

The CareBot system uses an Red–Green–Blue plus Depth (RGB-D) camera and Embedded GPU platform for edge AI (NVIDIA Jetson) computing module to enable on-device perception suitable for real-time operation in clinical environments. A key research challenge is enabling the robot to accurately detect and classify interaction points such as elevator buttons, access readers, and door panels under varying lighting, reflections, occlusions, and diverse panel designs - while maintaining efficient performance within embedded hardware constraints.

This project contributes to that research direction by designing and evaluating a scalable

computer-vision workflow for interaction-point detection on embedded systems. The work includes dataset creation, model design and optimisation, evaluation on prototype hardware, and preparation for future extensions. The resulting system will serve as a foundation for autonomous hospital mobility and support the development of assistive robotic technologies.

1.2 Problem Definition

The Ambyon ONE robot currently lacks a reliable computer-vision system to detect and recognize human-interaction elements (e.g., elevator and door control interfaces) in hospital environments, preventing autonomous navigation and interaction with building infrastructure.

1.3 Project Objective

Develop and validate an efficient and scalable computer-vision workflow that enables a service-robot platform to autonomously detect and classify human-interaction elements in hospital environments using embedded hardware.

1.4 Research Question(s)

Main Research Question:
How can an efficient and scalable computer-vision workflow be designed and implemented to enable a service robot to detect and interpret human-interaction elements in hospital environments using embedded hardware?

Sub-questions:

1. What are the relevant human–interaction points in hospital environments that should be detected and classified by the computer-vision workflow?
2. What computer-vision methods and model architectures are most suitable for detecting and classifying human-interaction elements in service-robot environments?
3. How can a high-quality dataset be designed and constructed for achieving robust model performance?
4. Which model-optimization techniques support real-time inference on embedded hardware while preserving detection accuracy?
5. How can the workflow support scalable and efficient model improvements, including potential data collection and retraining during deployment?

1.5 Outline of the report

Chapter 1 introduces the background, motivation, and objectives of the project. Chapter 2 reviews relevant literature on interaction-point perception, instance segmentation, and embedded computer-vision systems. Chapter 3 describes the methodology, including dataset preparation,

annotation strategy, model training, evaluation, and deployment considerations. Chapter 4 presents the experimental results and quantitative evaluation. Chapter 5 discusses the findings and their implications, and Chapter 6 concludes the report and outlines future work.

2 Literature Review

This chapter reviews the relevant scientific literature used in the Project, related to interaction-point perception in hospital environments, with a focus on instance segmentation, dataset design, annotation strategies, training and deployment of computer-vision models on embedded systems.

2.1 Perception of human–machine interaction elements

Human–machine interaction elements such as elevator buttons, door panels, and access readers are important targets for indoor service robots operating in hospital environments. Previous work in healthcare and service robotics shows that reliable interaction with such infrastructure is required for autonomous operation across multiple floors and restricted areas [6, 14].

From a computer-vision perspective, these interaction elements are challenging to detect because they are small, visually similar, and often arranged closely together on control panels. Studies on visual perception for robotic interaction report that elevator panels typically contain many buttons with minimal visual differences, which makes robust localization difficult under varying lighting conditions, reflections, and viewing angles [2].

General indoor-robot perception systems often focus on detecting large objects or structural elements and assume that interaction with building infrastructure is externally supported or environment-specific. However, research on autonomous service robots emphasizes that scalable deployment requires perception systems that can directly identify actionable interface elements from camera and sensor data, without relying on prior knowledge of the environment [21].

These findings indicate that interaction-point perception is a fine-grained vision problem in which individual interface elements must be localized accurately within visually dense scenes. This observation motivates the need for vision methods that go beyond coarse detection of panels or regions and instead support precise identification of individual interaction elements.

2.2 Formulating interaction-point perception as a vision problem

Vision-based perception problems in robotics are commonly formulated as image classification, object detection, or semantic segmentation tasks. Each formulation implies different assumptions about the structure of the scene and the required output, which directly affects suitability for robotic interaction.

Image classification assigns a single label to an entire image and is therefore unsuitable for scenarios where multiple interaction elements appear simultaneously. Object detection extends this formulation by predicting bounding boxes and class labels for individual objects, in the case of the project those are elevator buttons. While detection-based approaches have been widely used for indoor perception tasks, bounding boxes provide only coarse spatial localization and do not capture the precise shape of interaction elements [17, 16].

Semantic segmentation addresses this limitation by assigning a class label to each pixel in the

image. Formally, semantic segmentation can be expressed as a pixel-wise classification function

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \{1,\dots,K\}^{H\times W},$$

where H and W denote image height and width, C the number of input channels, and K the number of semantic classes [12]. Although this formulation enables precise localization, all pixels belonging to the same class are treated as a single region. As a result, semantic segmentation cannot distinguish between multiple instances of the same object class.

For interaction-point perception, this limitation is critical. Elevator panels often contain many visually similar buttons, each corresponding to a distinct action. Semantic segmentation assigns the same class label to all pixels belonging to a given category and therefore cannot distinguish between multiple instances of the same object class [12, 5]. As illustrated in Figure 2.1, a semantic segmentation output would label all button pixels identically, without separating individual buttons. This lack of instance-level differentiation makes semantic segmentation unsuitable for downstream decision-making and physical interaction, where each button must be treated as a separate actionable element.

Instance segmentation extends semantic segmentation by jointly performing object detection and pixel-wise classification, producing a separate mask for each object instance. This formulation can be expressed as a set of instance masks

$$\mathcal{M} = \{M_1, M_2, \dots, M_N\},$$

where each $M_i \in \{0,1\}^{H\times W}$ represents a binary mask for one object instance [5]. Instance segmentation therefore enables both precise spatial localization and differentiation between multiple objects of the same class. Figure 2.1 illustrates the conceptual differences between image classifi-

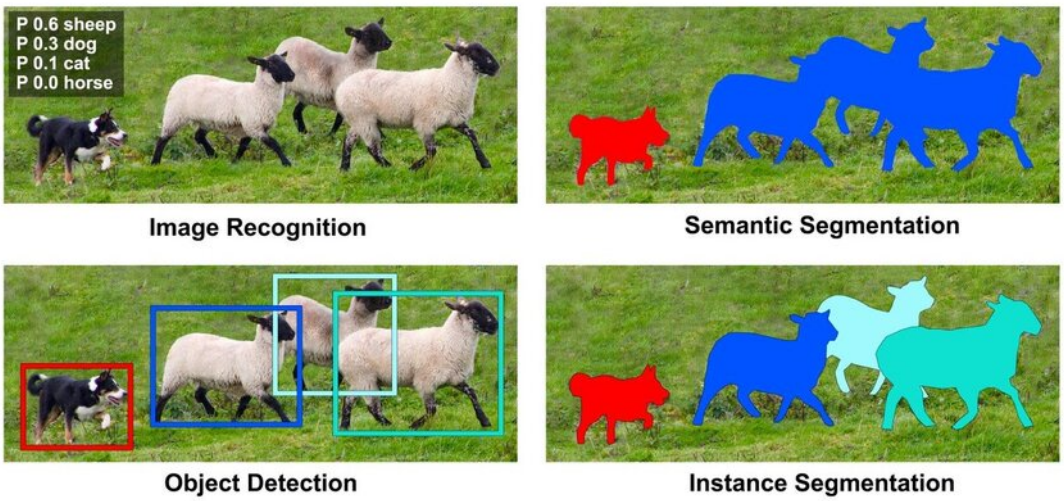


Figure 2.1: Comparison of image classification, object detection, semantic segmentation, and instance segmentation.

cation, object detection, semantic segmentation, and instance segmentation for visual perception tasks.

Based on findings in the literature and the requirements of interaction-point detection, in-

stance segmentation provides the most appropriate problem formulation for this project. It allows individual buttons to be identified as distinct entities while preserving pixel-level accuracy, which is essential for reliable robotic interaction in dense and visually complex control interfaces.

2.3 Public datasets and annotation strategies.

Public datasets for vision-based detection of elevator interfaces are limited, as most indoor robotics datasets focus on navigation or scene understanding rather than interaction elements [19]. This limits their direct applicability to interaction-point perception tasks.

The IRSO 2018 dataset provides Red–Green–Blue (RGB) images of elevator panels captured in realistic indoor environments and was originally introduced for object detection tasks [13]. Due to its focus on real elevator interfaces and visual diversity, it is well suited for studying interaction-point perception in service robotics.

Recent work reports a large-scale dataset with pixel-wise annotations for elevator button segmentation [11]. However, despite being described in the literature, the corresponding semantic segmentation dataset is not publicly available at the time this project is made and therefore cannot be used.

To enable instance-level interaction-point detection, this work reuses images from the IRSO 2018 dataset and replaces the original bounding box annotations with instance segmentation masks. Annotations are stored in the Common Objects in Context (COCO) format, which supports instance-level labeling and is widely used across computer vision frameworks, facilitating reuse and future extensions [10].

2.4 Model design principles, evaluation metrics, and semantic interpretation

Instance segmentation models for robotic perception typically follow a modular design in which feature extraction, object localization, and pixel-level mask prediction are handled by separate components within a single network. This design enables shared visual representations while supporting accurate instance-level localization, which is required for interaction-point perception in dense control interfaces.

Region-based instance segmentation models such as Mask Region-Based Convolutional Neural Network (Mask R-CNN) achieve high localization accuracy by explicitly separating region proposal, classification, and mask prediction stages [5]. Due to this design, Mask R-CNN is widely used as a reference architecture and benchmark model in instance segmentation research. However, the multi-stage nature of the model results in high computational cost and memory usage, which limits its suitability for real-time deployment on embedded robotic platforms.

To address these constraints, more lightweight instance segmentation architectures have been proposed that integrate detection and mask prediction into a single-stage pipeline. You Only Look Once (YOLO)-based segmentation models, such as YOLOv8-Seg, trade a moderate reduction in segmentation accuracy for substantially improved inference speed and computational

efficiency [8]. These models have demonstrated competitive performance in real-time and embedded settings, making them suitable candidates for deployment on resource-constrained robotic systems.

Model performance is commonly evaluated using overlap-based metrics that measure agreement between predicted masks and ground-truth annotations. Intersection over Union (IoU) is defined as

$$\text{IoU} = \frac{|M_p \cap M_{gt}|}{|M_p \cup M_{gt}|},$$

where M_p and M_{gt} denote the predicted and ground-truth masks. Average Precision (Average Precision (AP)), computed over multiple IoU thresholds, is widely used as a standard evaluation metric for instance segmentation tasks [10].

In many perception pipelines for elevator interfaces, instance localization is followed by a semantic interpretation stage that determines the functional meaning of each detected button. This is commonly achieved using Optical Character Recognition (OCR) or dedicated symbol classification models applied to the isolated button regions [11]. Accurate instance segmentation is a critical enabling step for such approaches, as it provides clean, well-aligned inputs for subsequent classification or text recognition.

In this work, instance segmentation is treated as the primary perception task, as reliable instance-level localization is a prerequisite for any semantic interpretation of interaction elements. The resulting pipeline is therefore designed to support integration of OCR or custom button classifiers, enabling the robot to associate detected buttons with their intended functions in downstream processing stages.

2.5 Embedded deployment constraints in robotic perception

Vision-based perception systems deployed on mobile service robots must operate under strict computational and energy constraints. Prior work in robotic vision reports that deep-learning models deployed on embedded platforms are limited by onboard memory, processing power, and power consumption, which directly affects real-time performance [3].

Studies evaluating deep-learning inference on NVIDIA Jetson-class hardware show that models achieving high accuracy on desktop Graphics Processing Units (GPUs) often exhibit increased latency and memory usage when deployed on embedded devices [9]. These findings indicate that benchmark accuracy alone is not a sufficient criterion for model selection in robotic perception systems.

For instance segmentation tasks, this trade-off is particularly relevant, as mask prediction introduces additional computational overhead compared to bounding-box detection. Research on efficient model design highlights that architectural choices such as backbone depth and feature reuse have a significant impact on inference efficiency [20]. Based on these insights, this project treats model efficiency and deployability as primary design constraints alongside segmentation accuracy.

Consequently, instance segmentation models considered in this work are evaluated not only with respect to localization performance, but also in terms of their suitability for embedded deployment. This literature-driven perspective directly informs the model selection and opti-

mization strategies described in Chapter 3(Methodology).

2.6 Scalability, robustness, and reproducibility.

Several studies emphasize that robust perception systems require more than a single well-performing model. Instead, scalability is achieved through structured data management, consistent annotation practices, and reproducible training pipelines that support incremental dataset expansion and model updates [4]. These practices reduce sensitivity to domain-specific bias and enable adaptation to new environments without redesigning the entire system.

From these findings, this project adopts a pipeline-oriented approach in which dataset creation, annotation, training, and evaluation are treated as modular and repeatable processes. Rather than optimizing a single model for a fixed dataset, the focus is placed on maintaining a perception workflow that can be extended with new data and retrained as deployment conditions evolve.

2.7 Summary and identified research gap

The reviewed literature shows that reliable perception of interaction elements in robotic systems requires instance-level localization, suitable annotation strategies, and awareness of embedded deployment constraints. Existing studies demonstrate the technical feasibility of these components, but often address them in isolation.

At the same time, publicly available datasets for elevator interfaces are limited, and existing annotations are commonly designed for object detection rather than segmentation tasks. In addition, many approaches focus on model performance without explicitly considering scalability, reproducibility, or deployment on embedded robotic platforms.

This project addresses these gaps by repurposing a realistic elevator dataset with instance-level annotations and by developing a scalable and deployment-aware computer-vision pipeline for interaction-point perception in hospital environments. The following chapter describes the methodology used to realize this approach.

3 Methodology

This chapter describes the methodology used to develop a computer-vision pipeline for interaction-point perception in hospital environments. The project is conducted with the objective of building a scalable and deployment-oriented perception system that can be iteratively improved across different hospital settings and reused within a robot fleet.

3.1 Overview of the proposed perception pipeline

The proposed methodology focuses on instance-level localization of interaction elements such as elevator buttons and control panels. Instance segmentation is used to detect individual button instances at pixel level, allowing precise localization and separation of closely spaced elements. This capability provides the basis for subsequent interpretation of button functions and autonomous interaction.

The methodology is designed to support an iterative training and deployment workflow. An initial base model is deployed on the robot, where it performs interaction-point perception and collects visual data during operation. Selected data can be transferred to an offboard system, where annotations may be reviewed, corrected, or extended before retraining the perception models. The updated models are then redeployed to the robot and become the new default for continued operation.

This workflow enables gradual adaptation to new hospital environments and allows knowledge acquired in one deployment to be reused and refined in subsequent deployments. The target end-to-end training and deployment process toward which this project is developed is illustrated in Figure 3.1.

The following sections describe each stage of the methodology in detail, beginning with dataset selection and preparation.

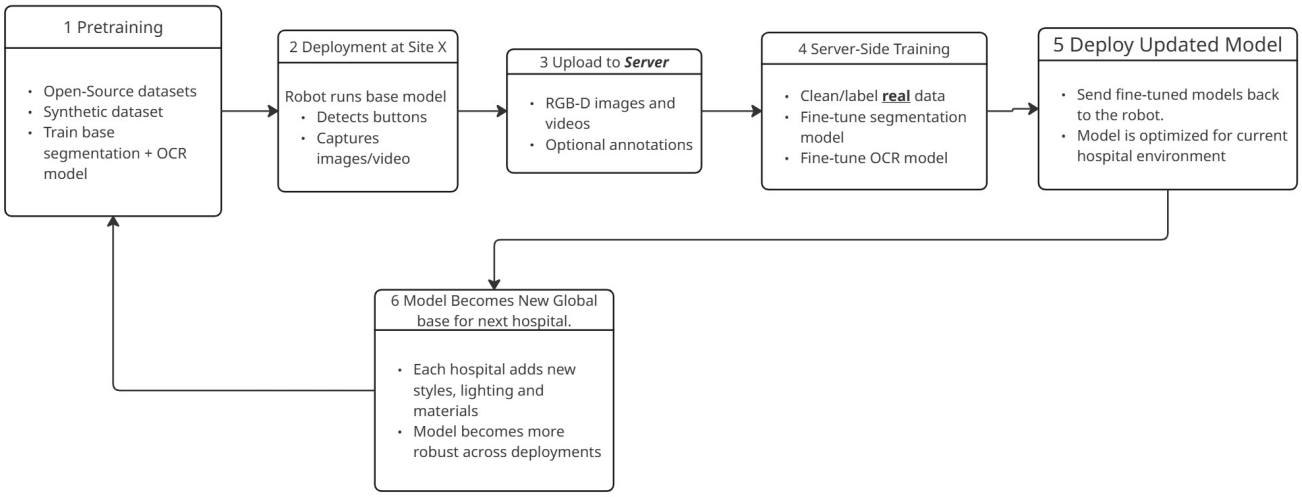


Figure 3.1: Target workflow for scalable training and deployment of interaction-point perception models across multiple hospital environments.

3.2 Dataset selection and preparation

This project uses a dataset derived from an established elevator interaction-point dataset. The original split used for this thesis is the repository-specific `elevator_split`, which is prepared in YOLO-compatible format for training instance-segmentation models. This section corresponds to the initial model-development step in Figure 3.1, detailing the dataset and its preparation before the first training cycle. The origin and relevance of this dataset were discussed in Literature Review; therefore, this section focuses on its structure and preparation for use in this project.

The initial dataset used for the baseline YOLOv8-Seg model was split as follows (`elevator_split`):

- train: 1122 images
- validation: 281 images
- test: 602 images (held out for final evaluation only)

This 1,400-image base subset (1122+281) was used to train the initial baseline model. The 602-image test split was never used for optimization or model selection.

In later stages, the original dataset was extended with images collected in the target installation (HAN University). For ADC adaptation, these base images were extended with real-environment images and re-split as the mixed dataset (`elevator_adc_mix`):

- train: 1154 images
- validation: 288 images
- test: 609 images

This mixed split was used for stage-wise tuning and the final ADC fine-tuning experiments. Each subset follows the same directory structure, containing an `images` folder with the raw RGB images and an annotation folder with matching labels.

In the original dataset, annotations are used in the project-specific YOLO-compatible instance-mask format. For the domain adaptation experiment, additional annotations from the target environment (ADC) were provided in COCO-style JSON and converted to YOLO-segmentation labels with a dedicated conversion script. This ensures compatibility with YOLO while maintaining a consistent train/val/test folder layout and class consistency.

The resulting dataset structure provides a consistent and reusable foundation for annotation, model training, and evaluation, as described in the following sections.

In addition to the instance-segmentation datasets, a recognition dataset for the semantic interpretation stage was derived from the same annotated source images. For this purpose, each annotated button instance was cropped from the original image using its existing bounding-box annotation, with an additional contextual padding of 15% of the box width and height. This produced a button-centered crop dataset for OCR fine-tuning without requiring any additional manual image collection.

The OCR dataset builder grouped crops by source image before creating a fixed 80/20 train/validation split with a deterministic random seed. As a result, all button crops originating

Figure 3.2: Example raw image from the dataset.

from one panel image were placed either in the training set or in the validation set, but never in both. This prevents information leakage caused by highly similar button appearances within the same elevator panel. The held-out OCR test set was generated directly from the original test images and their annotations.

Several OCR dataset variants were explored during development. The final variant used for the main recognition experiments removed labels named exactly `text` and labels starting with the prefix `text_`. These labels do not encode the actual button content, but merely indicate the presence of unspecified free text. Retaining them caused the recognizer to over-predict generic "TEXT"-like outputs and reduced generalization to unseen textual button content. The final OCR dataset, `ocr_rec_dataset_drop_textprefix_full`, contained 11,953 training crops, 2,973 validation crops, and 6,861 held-out test crops.

3.3 Annotation strategy and workflow

Instance-level ground truth was created using a semi-automatic workflow that combines model-based pre-annotation with manual verification. This strategy was used to efficiently generate high-quality instance segmentation annotations while maintaining consistency across the dataset.

Annotations were created in Computer Vision Annotation Tool (CVAT) [18], deployed locally using Docker [15] within a Windows Subsystem for Linux 2 (WSL2)-based Linux environment on a Windows host. Automatic annotation was enabled through CVAT–Nuclio integration, where the model was deployed as a Nuclio serverless function [7]. The instance segmentation model used for automatic annotation was Mask R-CNN [5].

During automatic annotation, CVAT sends task images to the Nuclio function, where Mask R-CNN predicts binary masks for each detected button instance. These masks are converted into polygon shapes within the Nuclio function and returned to CVAT as annotation data. The overall workflow used in this project is shown in Figure 3.3.

To support iterative improvement, the dataset was annotated in batches of approximately 150 images. After each batch, the model was retrained using the newly annotated data and reused for subsequent annotation rounds. Model checkpoints were created after approximately 100, 450, and 900 annotated images, enabling progressively improved pre-annotations and reduced manual correction effort.

All automatically generated annotations were also manually reviewed and corrected when

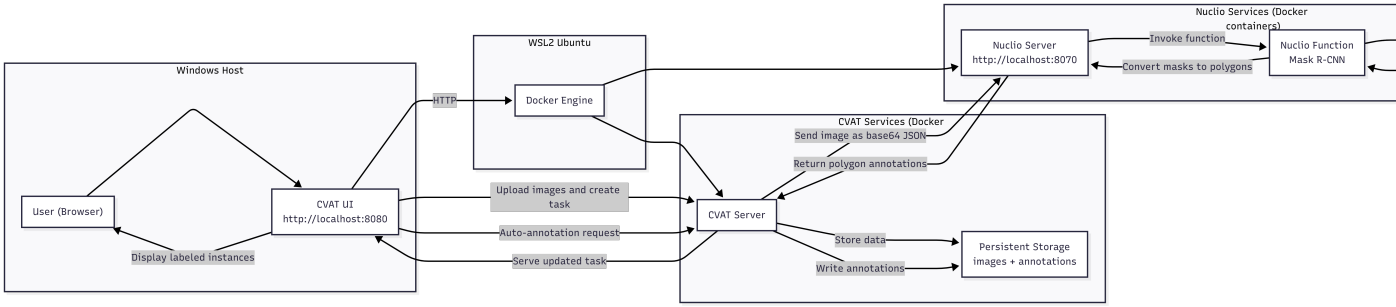


Figure 3.3: Semi-automatic annotation workflow on a Windows host using WSL2 and Docker. CVAT sends images to a Nuclio function running Mask R-CNN, which returns polygon instance annotations derived from predicted binary masks.

needed to ensure accurate instance boundaries, correct separation of adjacent buttons, and consistent labeling across both the training and test sets.

Figure 3.4: Example of an annotated image showing instance segmentation masks.

3.4 Dataset quality assessment

A systematic dataset quality assessment was performed to check whether image-level quality issues could bias model learning.

3.4.1 Statistical quality metrics.

For every training image, the following metrics were computed:

- **Sharpness**, estimated using the variance of the Laplacian:

$$\sigma^2_{\text{lap}} = \text{Var}(\nabla^2 I) \tag{3.1}$$

  where I is the grayscale image.

- **Brightness**, defined as the mean grayscale intensity.

- **Edge density**, defined as the fraction of pixels with gradient magnitude above the 90th percentile.

The analysis did not reveal a severe low-quality subset that should be removed. Low-score images were often hard but valid samples, so no global quality-based pruning was applied.

3.4.2 Model-assisted image assessment.

In addition to image-level quality analysis, a model-assisted approach was used to identify potentially harmful training samples.

Method. A trained instance segmentation model was used to evaluate each training image individually. For each image, the following metrics were computed:

- Mean Intersection over Union:

$$\text{IoU} = \frac{|M_{\text{pred}} \cap M_{\text{gt}}|}{|M_{\text{pred}} \cup M_{\text{gt}}|} \tag{3.2}$$

- Mean detection confidence:

$$\bar{c} = \frac{1}{N} \sum_{i=1}^{N} p_i \tag{3.3}$$

  where p_i is the softmax score of the i-th predicted instance.

- Number of false positives and false negatives.

Images were ranked using a composite criterion favoring low IoU, high false positives, and high confidence incorrect predictions. Based on those criteria, a harm-score is measured, based on which training set images are filtered and the potentially harmful ones are selected for manual inspection.

Inspection. Manual inspection of the lowest-ranked samples revealed two categories:

- Hard but valid images (extreme viewpoints, zoom levels, or geometric distortions).
- Negative images without buttons, occasionally producing false positives.

After manual inspection, removing hard but valid examples reduced recall. In contrast, excluding a small number of clear false-positive-only negatives had only marginal effect. The final training policy therefore used the full dataset with minimal exclusions.

3.5 Model architectures and training strategy

Two instance segmentation model families are employed in this project, each serving a distinct role within the overall perception pipeline. Mask R-CNN is used as a high-accuracy reference model and as the backbone of the semi-automatic annotation workflow (Section 3.3), while YOLOv8-Seg is trained and evaluated as a deployment-oriented solution suitable for embedded hardware.

Experimental hardware and software environment

All YOLO and OCR training experiments reported in this thesis were executed on the same workstation. The host system ran Ubuntu 22.04.5 LTS with Linux kernel 6.8.0-90. The machine was equipped with an AMD Ryzen 9 9950X 16-core processor, 176 GiB of system memory, and

an NVIDIA GeForce RTX 4090 graphics card with 24 GiB of Video Random Access Memory (VRAM). The training software environment used Python 3.10.12 and PyTorch 2.5.1 compiled against CUDA 12.1. This configuration provided the computational platform for the baseline training, hyperparameter tuning, OCR fine-tuning, and the comparative evaluation runs discussed in Chapters 3 and 4.

Common training strategy

All models are trained using a transfer learning approach. Networks are initialized from pretrained checkpoints and fine-tuned on the project-specific dataset, rather than being trained from scratch. This strategy improves convergence speed and generalization performance, particularly given the moderate dataset size.

The final mixed dataset used for adaptation fine-tuning follows a fixed random split (seed 123) with train/validation/test separation for model selection and reporting. Standard augmentation is applied during training (HSV jitter, geometric perturbations, and horizontal flip with $p = 0.5$) to improve robustness to viewpoint and lighting variations.

For the initial baseline training phase, training used only the available train split and the test split was held out for final validation.

Mask R-CNN training

Mask R-CNN is trained using a custom PyTorch-based pipeline with COCO-formatted instance annotations. Training is performed for 50 epochs with a batch size of 2, reflecting the higher memory requirements of region-based instance segmentation models.

Optimization is performed using the AdamW optimizer with a learning rate of 5×10^{-4} and weight decay of 1×10^{-4}. A ReduceLROnPlateau learning rate scheduler is employed, reducing the learning rate by a factor of 0.5 when the validation loss does not improve for three consecutive epochs. Early stopping is enabled with a patience of 10 epochs based on validation loss.

Mask R-CNN is retrained iteratively as the annotated dataset grows. This iterative training is applied exclusively to the Mask R-CNN model, as it is used to generate pre-annotations during the semi-automatic annotation process. Model checkpoints are created after approximately 100, 450, and 900 annotated training images, enabling progressively improved annotation quality and reduced manual correction effort.

YOLOv8-Seg training

YOLOv8-Seg is trained using the Ultralytics framework with the pretrained `yolov8s-seg.pt` checkpoint. The initial baseline configuration was trained for 100 epochs with an input resolution of 640×640, dynamic batch sizing (`batch=-1`), seed 123, warmup over 5 epochs, and early stopping patience of 10 epochs. The augmentation policy followed the standard Ultralytics recipe used throughout this study, including HSV perturbation, small geometric transformations, horizontal flipping, and mosaic augmentation.

Although this baseline produced strong results, its training settings were selected empirically. A structured hyperparameter tuning procedure was therefore conducted to determine whether

performance could be improved in a more systematic and reproducible way. All tuning runs were logged and tracked in Weights & Biases (W&B), which was used to store the hyperparameter configurations, training curves, and validation metrics of individual experiments.

The three stages of the hyperparameter tuning procedure are summarized in Table 3.1. The table is intended to provide a compact overview of the explored search spaces and stopping criteria, while the accompanying text explains the rationale behind each stage.

Table 3.1: Three-stage hyperparameter tuning procedure for YOLOv8-Seg.

Stage	Purpose	Tuned or varied settings	Training setup	Selection rule
Stage 1	Narrow hyperparameter search around the baseline recipe	`lr0`, `lrf`, `weight_decay`, `momentum`, `hsv_h`, `hsv_s`, `hsv_v`, `degrees`, `translate`, `scale`, `mosaic`, `fliplr`, `warmup_epochs`, `close_mosaic`	24 trials, 30 epochs each; sanity run of 12 epochs beforehand	Top runs by validation Mean Average Precision (mAP)
Stage 2	Capacity sweep using the best Stage 1 configurations	Batch size $\in \{8, 16\}$, image size $\in \{640, 768\}$; Stage 1 parameters kept fixed	100 epochs, patience 15	Top 5 Stage 1 runs with $\text{mAP}^{(M)}_{50:95} \geq 0.30$
Stage 3	Late-stage refinement from the best Stage 2 checkpoints	Resume from best checkpoint, halve `lr0`	180 epochs	Top 2 Stage 2 runs with $\text{mAP}^{(M)}_{50:95} \geq 0.50$

The tuning process was organized into three stages. In the first stage, a narrow hyperparameter search was performed around the known-good baseline configuration using Ray Tune within the Ultralytics framework. Twenty-four trials were executed, each for 30 epochs, while the dataset split remained fixed so that the validation set was used for model selection and the test set remained untouched. The tuned hyperparameters were the initial and final learning-rate factors (`lr0`, `lrf`), `weight_decay`, `momentum`, color augmentation parameters (`hsv_h`, `hsv_s`, `hsv_v`), `degrees`, `translate`, `scale`, `mosaic`, `fliplr`, `warmup_epochs`, and `close_mosaic`. To preserve comparability with the baseline recipe, `optimizer=SGD`, `mixup=0`, `shear=0`, and `perspective=0` were kept fixed. Before the full sweep, a short 12-epoch sanity run was executed to confirm that the training pipeline, dataset configuration, and logging setup behaved as expected.

The second stage focused on capacity-related settings rather than optimization or augmentation parameters. The five best-performing configurations from Stage 1 with $\text{mAP}^{(M)}_{50:95} \geq 0.30$ were selected, and each of them was retrained while varying only batch size and input resolution. The tested combinations were batch sizes of 8 and 16, and image sizes of 640 and 768 pixels. Each run was trained for 100 epochs with patience 15, while all tuned Stage 1 hyperparameters were inherited unchanged. This design allowed the influence of computational capacity settings to be isolated from the already tuned optimization parameters.

The third stage served as a refinement step. The two best Stage 2 configurations with $\text{mAP}^{(M)}_{50:95} \geq 0.50$ were resumed from their best saved checkpoints and trained for 180 additional

epochs. In this phase, the initial learning rate was halved to support more conservative late-stage optimization.

Two additional experiments were conducted after the three-stage hyperparameter tuning process. First, a YOLO26-small segmentation model was trained using the same baseline training recipe, followed by a longer 180-epoch run to assess whether the newer architecture could outperform YOLOv8-Seg under comparable settings. Second, the best YOLOv8 baseline checkpoint was fine-tuned on the mixed `elevator_adc_mix` dataset to evaluate domain adaptation with data collected from the target environment by the Active Data Collector (ADC). This adaptation run was trained for 40 epochs using conservative settings (`lr0=0.001`, `patience=0`) to improve environment-specific performance without overwriting the original baseline model.

Across the tuning and fine-tuning stages, the train/validation split was used for model fitting and model selection. For reporting and model comparison, held-out test splits were used only for final evaluation.

Loss functions and optimization objectives

Both instance segmentation models are trained by minimizing a composite loss function that jointly optimizes object classification, localization, and mask prediction. Although Mask R-CNN and YOLOv8-Seg differ architecturally, their optimization objectives are conceptually aligned.

Mask R-CNN. The training objective of Mask R-CNN is defined as a multi-task loss composed of three terms [5]:

$$\mathcal{L}_{\text{Mask R-CNN}} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}, \tag{3.4}$$

where $\mathcal{L}_{\text{cls}}$ denotes the classification loss, implemented as categorical cross-entropy; $\mathcal{L}_{\text{box}}$ represents bounding-box regression using the smooth L_1 loss; and $\mathcal{L}_{\text{mask}}$ is a pixel-wise binary cross-entropy loss applied to each predicted instance mask.

YOLOv8-Seg. YOLOv8-Seg integrates detection and segmentation into a single-stage architecture. The total training loss is expressed as a weighted sum of classification, localization, and segmentation losses [8]:

$$\mathcal{L}_{\text{YOLO}} = \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{seg}}\mathcal{L}_{\text{seg}}, \tag{3.5}$$

where $\mathcal{L}_{\text{seg}}$ represents the segmentation loss applied to the predicted mask outputs, and the weighting coefficients λ control the relative contribution of each task during optimization.

Despite architectural differences, both models optimize comparable objectives, enabling consistent evaluation of segmentation accuracy and computational efficiency across reference and deployment-oriented architectures.

OCR-based button interpretation

Following instance segmentation, the detected button regions are passed to a semantic interpretation stage. Segmented button instances are cropped and processed by an OCR-based recognition module to extract character labels or symbols associated with each button. This

stage enables the robot to associate detected interaction points with their functional meaning and supports downstream decision-making during autonomous operation.

The semantic interpretation stage is formulated as a cropped-image recognition problem rather than as full-scene text detection. This design choice follows directly from the preceding instance segmentation stage: once a button instance has already been localized, the recognition model no longer needs to solve the harder problem of finding text within the full image. Instead, it receives a tightly bounded crop around a single button and predicts the button's semantic label. This reduces computational cost and makes the OCR stage more suitable for embedded deployment.

Recognition model architecture. For cropped-button recognition, a lightweight OCR recognizer based on `PP-OCRv5_mobile_rec` was selected. This model was preferred because it is designed for efficient deployment and already provides a compact recognition backbone suitable for embedded hardware. In simplified form, the recognizer consists of a compact visual feature extractor, a sequence-modeling stage that arranges features in reading order, and a recognition head that predicts the final label string. In the selected configuration, the feature extractor is based on `PPLCNetV3` with scale 0.95, meaning that a slightly reduced-width version of the backbone is used to keep the model compact. The sequence-modeling stage is `SVTR`-based, and the recognition head combines `CTC` and `NRTR` objectives during training. This architecture is appropriate for the current task because button labels range from single digits and single letters to short strings such as `10`, `4A`, or `CLOSE`, while still requiring efficient inference on cropped inputs.

Only the recognition component of the OCR stack was used. The detection component of Paddle Optical Character Recognition (PaddleOCR) was intentionally omitted from the final recognition pipeline, because button localization was already solved upstream by the instance segmentation model. This avoids redundant computation and makes the OCR stage conceptually cleaner: segmentation localizes the button instance, and OCR interprets that localized instance.

Fine-tuning strategy. The recognizer was initialized from the official pretrained English `PP-OCRv5_mobile_rec` weights and fine-tuned on the project-specific crop dataset. During training, each crop was resized to the fixed input resolution of 48×320 defined in the PaddleOCR recognition configuration so that all samples reached the network with a consistent image height and width. Optimization was performed with Adam, cosine learning-rate scheduling, warmup in the early training phase, and L2 regularization. In this context, cosine learning-rate scheduling means that the step size starts relatively high and then decreases smoothly during training, allowing larger parameter updates in the early epochs and more conservative refinement in the later epochs.

Recognition-specific data augmentation and multiscale sampling were enabled to improve robustness to appearance variation and to reduce overfitting. In practice, this prevents the recognizer from seeing each button crop in only one fixed presentation and improves tolerance to realistic differences in scale, contrast, blur, and local appearance.

The implementation followed the standard PaddleOCR training workflow. Rather than building a custom OCR training loop from scratch, the official `tools/train.py` and `tools/eval.py`

scripts provided by the PaddleOCR framework were used directly. Project-specific experiments were defined through YAML Ain't Markup Language (YAML) configuration files and command-line overrides, in the same spirit as using configuration-driven training through Ultralytics for the YOLO experiments.

Model selection followed a two-stage hyperparameter tuning procedure. The first stage varied only the total number of epochs while keeping the baseline optimizer settings fixed. Epoch budgets of 30, 50, 60, and 80 were evaluated in order to determine whether the original 60-epoch recipe was necessary. This stage showed that the model reached its best validation performance before epoch 50 and that training beyond this point produced negligible gain. The second stage therefore fixed the epoch budget at 50 and performed a narrow sweep over optimizer hyperparameters, primarily learning rate and weight decay. The main configurations tested were $\{3 \times 10^{-4}, 5 \times 10^{-4}, 7 \times 10^{-4}\}$ for the initial learning rate and $\{1 \times 10^{-5}, 3 \times 10^{-5}\}$ for weight decay, while warmup was adjusted conservatively between 3 and 5 epochs. All OCR tuning runs were logged in W&B so that training curves, hyperparameter choices, and validation metrics could be compared systematically.

3.6 Evaluation metrics and methodology

The performance of the instance segmentation models is evaluated using a combination of standard COCO metrics and task-oriented instance-level metrics. This dual evaluation strategy allows both standardized benchmarking and direct assessment of model suitability for robotic interaction tasks.

3.6.1 Intersection over Union

Segmentation quality is measured using Intersection over Union (IoU). For a predicted instance mask M_p and the corresponding ground-truth mask M_{gt}, IoU is defined as:

$$\text{IoU} = \frac{|M_p \cap M_{gt}|}{|M_p \cup M_{gt}|} \tag{3.6}$$

IoU values range from 0 (no overlap) to 1 (perfect overlap). In this project, IoU is computed at the instance level and used both for matching predicted and ground-truth instances and for computing summary statistics.

3.6.2 Precision, recall, and F1-score

Instance-level detection performance is quantified using precision, recall, and F1-score. A predicted instance is considered a true positive if its IoU with an unmatched ground-truth instance exceeds a threshold of 0.5. Precision and recall are defined as:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.7}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.8}$$

where TP, FP, and FN denote the number of true positives, false positives, and false negatives, respectively.

The F1-score, which balances precision and recall, is computed as:

$$\mathrm{F1} = \frac{2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}} \tag{3.9}$$

These metrics provide intuitive insight into detection reliability and are particularly relevant for robotic interaction tasks, where both missed detections and false positives can lead to failed interactions.

3.6.3 Average Precision under the COCO protocol

To enable standardized benchmarking, model performance is additionally evaluated using the COCO instance segmentation protocol. Average Precision (AP) is computed as the area under the precision–recall curve:

$$\mathrm{AP} = \int_0^1 p(r)\, dr \tag{3.10}$$

where $p(r)$ denotes precision as a function of recall r. Following the COCO evaluation methodology, AP is reported as the mean over multiple IoU thresholds ranging from 0.50 to 0.95 in increments of 0.05. Additional metrics such as AP_{50} and AP_{75} are also reported to provide insight into performance at specific overlap thresholds.

3.6.4 Mean IoU and runtime performance

For correctly matched instances, the mean IoU is computed across the test set to assess overall segmentation accuracy. In addition to accuracy metrics, runtime performance is measured by recording per-image inference time. Average inference time and frames per second (Frames Per Second (FPS)) are reported to evaluate suitability for real-time deployment on embedded hardware.

3.6.5 OCR accuracy and normalized edit distance

The OCR recognition models are evaluated using exact-match accuracy and normalized edit distance. Exact-match accuracy is the fraction of button crops for which the predicted label string matches the ground-truth label exactly after the same normalization procedure is applied to both. This metric is strict and is therefore suitable for the final semantic interpretation task, where predicting `4` instead of `14`, or `OPE` instead of `OPEN`, must be counted as an error.

$$\mathrm{Accuracy}_{\mathrm{OCR}} = \frac{N_{\mathrm{correct}}}{N_{\mathrm{total}}} \tag{3.11}$$

where N_{correct} is the number of exactly correct predictions and N_{total} is the total number of evaluated crops.

In addition to exact-match accuracy, normalized edit distance is used to quantify how close an incorrect prediction is to the target string. This metric is derived from the Levenshtein edit distance between prediction and target and is normalized to the interval $[0, 1]$, where 1 indicates

a perfect match. Unlike exact-match accuracy, normalized edit distance gives partial credit to near-correct predictions, which is useful when analyzing OCR behavior on short strings and icon labels that are often confused by only one or two characters.

$$\mathrm{NED} = 1 - \frac{1}{N}\sum_{i=1}^{N}\frac{d_{\mathrm{edit}}(\hat{y}_i, y_i)}{\max(|\hat{y}_i|, |y_i|, 1)} \tag{3.12}$$

where $\hat{y}_i$ is the predicted string, y_i is the corresponding ground-truth string, and d_{edit} denotes the Levenshtein edit distance.

During OCR tuning, validation accuracy was treated as the primary model-selection metric, while normalized edit distance was used as a secondary quality indicator. Final OCR benchmarking was then performed on the held-out OCR test split using the official PaddleOCR evaluation script with the same configuration family as used during training.

3.6.6 Evaluation protocol

All evaluations are conducted on a held-out test set that is not used during training or validation. Identical datasets, matching criteria, and thresholds are used across all model architectures to ensure fair comparison. COCO-based metrics are computed using the official evaluation tools, while precision, recall, F1-score, mean IoU, and runtime statistics are computed using a custom evaluation pipeline.

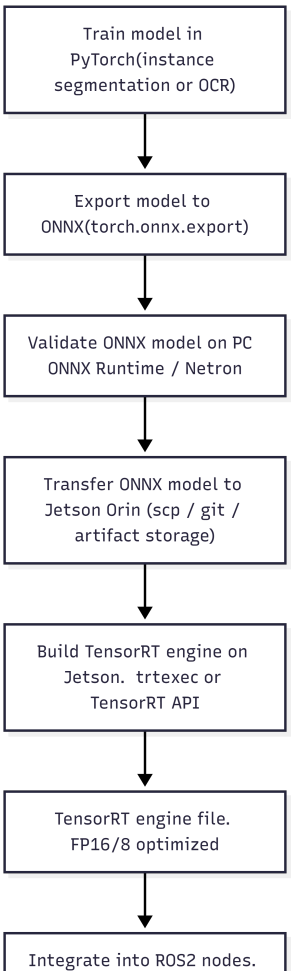
The evaluation methodology described in this section forms the basis for the quantitative comparison of models presented in Chapter 4 (Results).

3.7 Embedded deployment and optimization

To enable real-time execution on embedded hardware, and wrap up step 1 of the workflow strategy in Figure 3.1, trained models are optimized and deployed on a NVIDIA Jetson Orin platform. The deployment workflow is designed to be identical for both the instance segmentation model and the OCR-based button classification model.

Models are first trained in PyTorch and exported to the Open Neural Network Exchange (ONNX) [1] format. The exported ONNX models are validated on a desktop system using ONNX Runtime and model inspection tools to ensure numerical correctness and structural compatibility prior to deployment.

Validated ONNX models are then transferred to the NVIDIA Jetson Orin device, where TensorRT is used to build optimized inference engines. Engine generation is performed directly on the target device to ensure compatibility with the specific GPU archi-



tecture. Half-precision floating point format (FP16) precision is used to reduce memory footprint and improve inference speed while maintaining sufficient numerical accuracy.

The resulting TensorRT engines are integrated into Robot Operating System 2 (ROS 2) nodes responsible for perception tasks. At runtime, image data from the RGB-D camera is passed through the ROS 2 pipeline and processed by the TensorRT engines to produce instance segmentation masks and corresponding button labels.

The complete deployment and optimization workflow is illustrated in Figure 3.5. This approach ensures a consistent and efficient deployment pipeline and enables fair evaluation of different model architectures under identical runtime constraints.

4 Results

This chapter presents the quantitative results obtained for the main technical components of the proposed elevator-interaction pipeline. The current version of the report focuses on the visual detection subsystem based on YOLO instance segmentation and the OCR-based button interpretation stage. Unless stated otherwise, final model comparisons are reported on held-out test sets, while model selection during training and hyperparameter tuning is based on the validation split.

4.1 YOLO hyperparameter tuning and evaluation

4.1.1 Baseline YOLOv8-Seg performance

The baseline YOLOv8-Seg model trained on `elevator_split` established the main reference point for all subsequent experiments. On the validation split, the best checkpoint was obtained at epoch 49. This model achieved a mask mAP_{50} of 0.93931 and a mask $\text{mAP}_{50:95}$ of 0.59645, with mask precision and recall of 0.90623 and 0.89947, respectively. These results confirmed that the initial configuration was already strong enough to serve as a realistic benchmark rather than only as a preliminary starting point.

When evaluated on the held-out original test split at an image size of 640 pixels, the same baseline model achieved a mask mAP_{50} of 0.91921 and a mask $\text{mAP}_{50:95}$ of 0.58295. This test result is slightly lower than the best validation value, which is expected, but it still indicates good generalization to unseen elevator-panel images.

4.1.2 Hyperparameter tuning results

The three-stage hyperparameter tuning procedure improved validation performance, but the gains were moderate. Stage 1 served as a narrow Ray Tune search around the baseline configuration. As shown in Figure 4.1, most of the displayed Stage 1 trials did not produce meaningful mask performance. Among the first ten runs shown in the W&B export, only three configurations reached non-zero $\text{mAP}_{50:95}$ values, while the remaining trials stayed at zero throughout the 30-epoch search horizon. These zero-valued runs still count as valid tuning outcomes because they identify unsuccessful regions of the hyperparameter space. The main conclusion from Stage 1 was therefore not that tuning broadly improved all runs, but that only a small subset of configurations was suitable for further evaluation in Stage 2.

The most relevant outcome of the tuning pipeline was that the best Stage 2 configuration consistently selected a batch size of 16 and an input resolution of 768 pixels. Stage 3 then resumed training from this configuration with a reduced initial learning rate.

Figure 4.2 shows the broader Stage 2 validation mask $\text{mAP}_{50:95}$ trends. In contrast to Stage 1, Stage 2 produced a larger number of usable configurations because the search was restricted to the top Stage 1 candidates and only varied image size and batch size. The logged results show that eight out of sixteen unique Stage 2 configurations achieved non-zero mask $\text{mAP}_{50:95}$ values. Most successful runs were observed at an input resolution of 640 pixels, while many 768-pixel

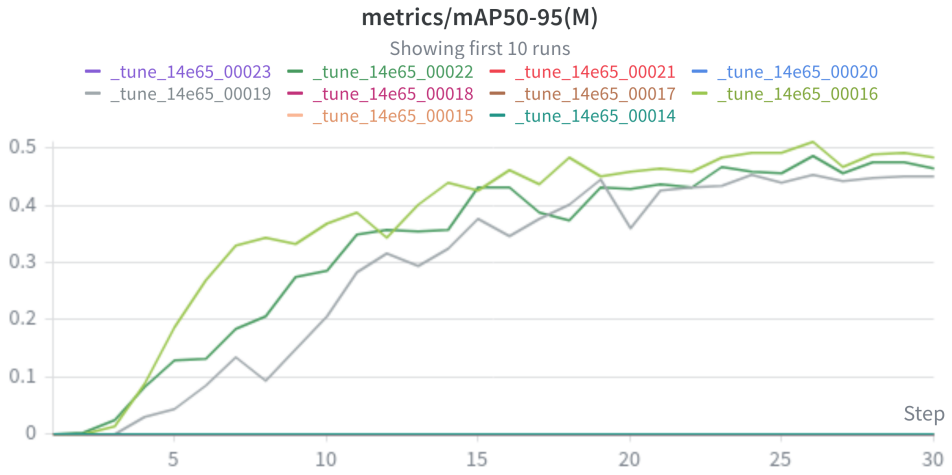


Figure 4.1: Stage 1 validation mask $mAP_{50:95}$ for the first ten runs shown in the W&B sweep export. Only a small subset of configurations produced non-zero validation performance.

variants remained unsuccessful. The main exception was the `cfg1_b16_sz768` configuration, which produced the best Stage 2 result and was therefore selected for Stage 3 refinement.

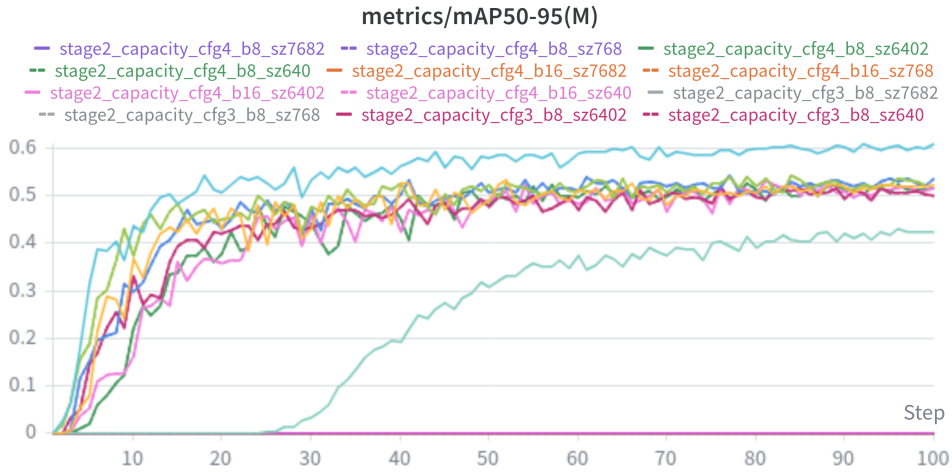


Figure 4.2: Stage 2 validation mask $mAP_{50:95}$ for the broader set of capacity-tuning runs. Compared with Stage 1, more configurations produced usable validation performance, although unsuccessful runs were still present.

Table 4.1 summarizes the best validation results of the baseline model and the best Stage 3 tuned model. The tuned model reached its best validation performance at epoch 120 and improved mask $mAP_{50:95}$ from 0.59645 to 0.63064. This corresponds to an absolute gain of 0.03419 on the validation set. Precision changed only slightly, while recall increased from 0.89947 to 0.90810, indicating that the tuned model mainly benefited from better localization and mask quality rather than from a major shift in confidence calibration. This improvement, however, should be interpreted as a validation-stage model-selection result and not as evidence that the tuned model outperformed the baseline on the final held-out test set.

Figures 4.3 and 4.4 show the behavior of the two Stage 3 refinement runs. Both runs

followed almost identical validation trajectories, which indicates that the refinement step was stable once the best Stage 2 configuration had been selected. The validation mask $mAP_{50:95}$ increased rapidly in the early epochs and then stabilized slightly above 0.62. At the same time, the validation segmentation loss dropped sharply at the start of training and then remained close to a stable plateau. These curves indicate a controlled refinement of an already strong configuration rather than a major qualitative change in optimization behavior.

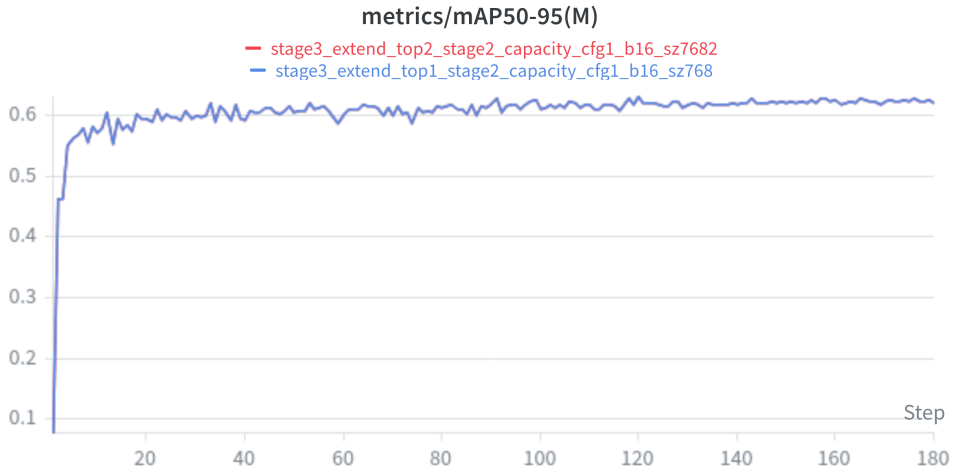


Figure 4.3: Stage 3 validation mask $mAP_{50:95}$ for the two refinement runs. The runs followed nearly identical trajectories and converged to the same performance region.

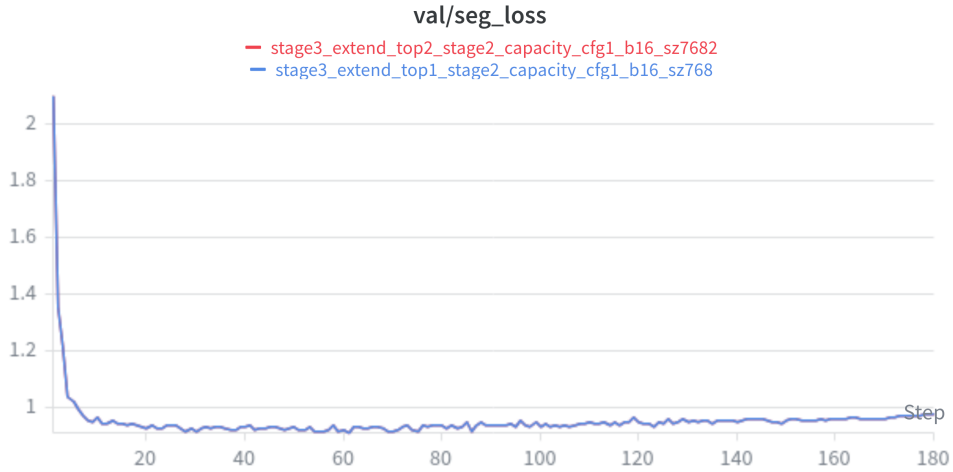


Figure 4.4: Stage 3 validation segmentation loss. After a steep initial decrease, the loss remained stable, indicating consistent convergence during refinement.

These results show that systematic hyperparameter tuning was useful for improving validation performance. However, the held-out test comparison in Table 4.2 does not support replacing the baseline checkpoint with the Stage 3 model. At an image size of 640 pixels, the baseline outperformed the tuned model by 0.04915 mask $mAP_{50:95}$, and at 768 pixels the margin remained 0.02767 in favor of the baseline. For this reason, the original baseline checkpoint remained the reference model for the later deployment-oriented experiments and for the final

Table 4.1: Best validation results of the baseline and tuned YOLOv8-Seg models.

Model	Best epoch	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	49	0.93931	0.59645	0.90623	0.89947
Stage 3 tuned YOLOv8-Seg	120	0.94268	0.63064	0.90150	0.90810

Table 4.2: Held-out test-set comparison between the baseline and the selected Stage 3 tuned YOLOv8-Seg model.

Model	Image size	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	640	0.92819	0.73744	0.87603	0.90832
Stage 3 tuned YOLOv8-Seg	640	0.91706	0.68829	0.86839	0.88443
Baseline YOLOv8-Seg	768	0.92638	0.74752	0.86778	0.92329
Stage 3 tuned YOLOv8-Seg	768	0.92719	0.71985	0.87399	0.90724

adaptation study.

4.1.3 Comparison with YOLO26

To evaluate whether a newer Ultralytics architecture could provide a better segmentation model under comparable conditions, YOLO26-small was trained using the same baseline recipe and later extended to a longer 180-epoch run. The final comparison was performed on the held-out original test split at image sizes of 640 and 768 pixels.

The results are presented in Table 4.3. At an image size of 640 pixels, the baseline YOLOv8-Seg model achieved a mask $mAP_{50:95}$ of 0.58295, whereas YOLO26 reached 0.57538. At 768 pixels, the baseline again remained slightly better, with 0.62325 compared with 0.61689 for YOLO26. The higher-resolution evaluation benefited both models, but it did not change the ranking between them. Under the training and evaluation conditions used in this project, YOLO26 therefore did not outperform the baseline YOLOv8-Seg model on the held-out test set.

Figure 4.5 shows the corresponding validation mask $mAP_{50:95}$ curves for the two YOLO26 runs. Extending the training to 180 epochs shifted the validation curve upward and produced a higher plateau than the shorter run. However, the improvement remained modest and was not sufficient to reverse the final test-set ranking against the baseline YOLOv8-Seg model.

Table 4.3: Test-set comparison between the baseline YOLOv8-Seg model and YOLO26-small.

Model	Image size	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
YOLOv8-Seg baseline	640	0.91921	0.58295	0.86835	0.89896
YOLO26-small	640	0.91209	0.57538	0.87300	0.89879
YOLOv8-Seg baseline	768	0.92384	0.62325	0.86577	0.91939
YOLO26-small	768	0.92596	0.61689	0.88408	0.91326

4.1.4 Environment-specific fine-tuning with ADC data

The final YOLO experiment focused on adaptation to the target university environment. For this purpose, the baseline checkpoint was fine-tuned on a mixed dataset that combined the original

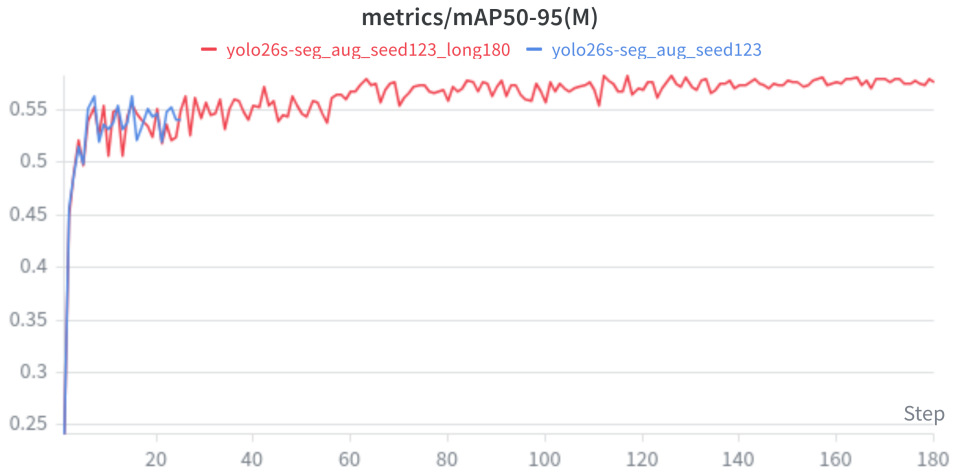


Figure 4.5: Validation mask $mAP_{50:95}$ for the two YOLO26 training runs. The longer 180-epoch run improved over the shorter run, but the resulting model still did not surpass the YOLOv8 baseline on the held-out test set.

elevator-panel dataset with 46 newly collected and annotated images from HAN University, Building 31. This building served as the pilot environment for planned robot deployment tests. The environment-specific images were collected with the ADC and then split into 32 training images, 7 validation images, and 7 test images before being merged with the original dataset. The resulting evaluation therefore makes it possible to determine whether the adapted model improves in the target environment without losing too much performance on the original domain.

Table 4.4 shows the results on the original held-out test split. The fine-tuned model remained effectively tied with the baseline. Its mask $mAP_{50:95}$ changed from 0.58295 to 0.58293, which is negligible. The box metrics improved slightly, and mask precision and recall also increased marginally. This indicates that adaptation to the new environment did not cause a meaningful degradation on the original dataset.

The corresponding validation curve is shown in Figure 4.6. The fine-tuned model remained in the same overall validation range as the original baseline, with modest fluctuations around 0.58. This is consistent with the final original-test comparison, which showed that the fine-tuned model preserved the baseline's general performance while becoming substantially more effective on the environment-specific `adc_first` test set.

Table 4.4: Performance on the original held-out elevator test split before and after ADC fine-tuning.

Model	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	0.91921	0.58295	0.86835	0.89896
ADC fine-tuned YOLOv8-Seg	0.91912	0.58293	0.88043	0.90050

The effect on the ADC-specific test set is much more pronounced, as shown in Table 4.5. On this environment-specific split, the baseline model achieved a mask $mAP_{50:95}$ of only 0.12290. After fine-tuning, the same metric increased to 0.50463. Mask precision increased from 0.61209 to 1.00000, and mask recall increased from 0.70136 to 0.99712. This is the clearest result of the

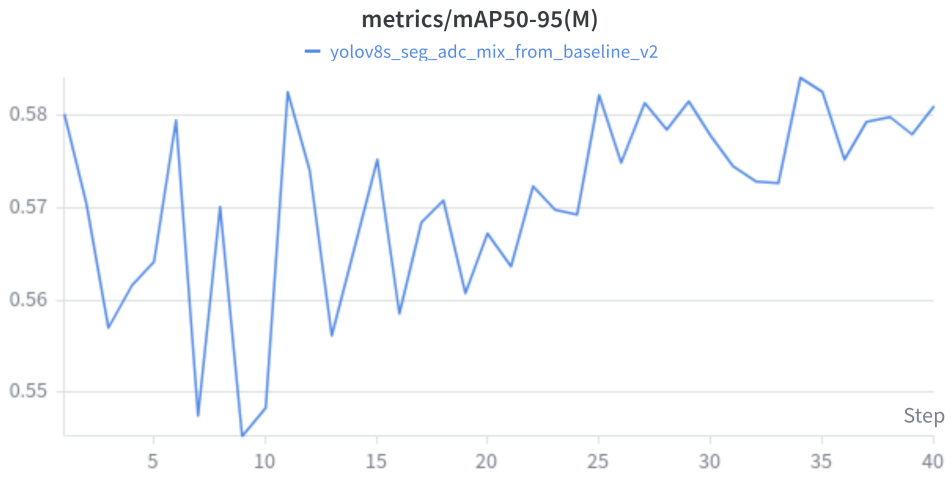


Figure 4.6: Validation mask $mAP_{50:95}$ during ADC-based fine-tuning. The curve remains close to the baseline validation range, which aligns with the near-identical performance observed on the original held-out test split.

YOLO experiments: collecting a relatively small amount of real deployment data and using it for controlled fine-tuning substantially improved performance in the target environment while preserving the performance level on the original test set.

Table 4.5: Performance on the environment-specific ADC test split before and after ADC fine-tuning.

Model	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	0.53520	0.12290	0.61209	0.70136
ADC fine-tuned YOLOv8-Seg	0.99500	0.50463	1.00000	0.99712

Overall, the YOLO results show three main findings. First, the baseline YOLOv8-Seg model was already strong and difficult to surpass on the general elevator test set. Second, staged hyperparameter tuning improved validation performance but did not fundamentally change the final model ranking. Third, the most practically valuable improvement was achieved through environment-specific fine-tuning with ADC data, which substantially increased performance on the target deployment domain while preserving performance on the original dataset.

4.2 OCR hyperparameter tuning and evaluation

The OCR experiments focused on improving cropped-button recognition while keeping the recognizer lightweight enough for later embedded deployment. As in the YOLO experiments, model selection during OCR tuning was based on the validation split, and the final comparison between OCR candidates was performed on the held-out OCR test set.

4.2.1 Stage 1: epoch selection

The first OCR tuning stage varied only the number of training epochs while keeping the optimizer configuration fixed. The main objective of this stage was to determine whether the original 60-

epoch fine-tuning recipe was necessary or whether a shorter training budget would already achieve the same validation quality.

Table 4.6 shows that the 50-epoch and 60-epoch runs converged to almost identical validation performance, both reaching their best result at epoch 46. In contrast, the 30-epoch run underperformed clearly. This indicates that the recognizer required more than 30 epochs to converge, but that training beyond 50 epochs provided no meaningful additional benefit. For that reason, 50 epochs were selected as the fixed budget for the next stage of OCR hyperparameter tuning. This interpretation is also supported by the Stage 1 training-loss curves shown in Figure 4.7, where the main decrease occurs well before the end of the 50-epoch run and the longer schedule does not suggest a qualitatively different convergence regime.

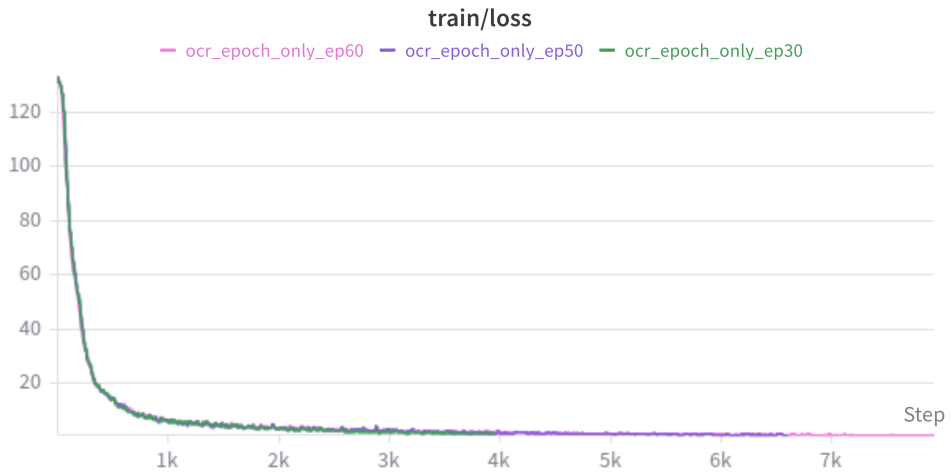


Figure 4.7: Stage 1 OCR training loss for the epoch-selection runs. Most of the optimization progress occurs before the 50-epoch mark, which supports the decision to use 50 epochs for the subsequent optimizer sweep.

Table 4.6: Stage 1 OCR epoch-sweep results on the validation split.

Trial	Best epoch	Accuracy	Normalized edit distance
30 epochs	16	0.78776	0.83732
50 epochs	46	0.84124	0.87080
60 epochs	46	0.84057	0.86893

4.2.2 Stage 2: learning rate and weight decay sweep

After fixing the training budget at 50 epochs, a second tuning stage evaluated a narrow optimizer sweep over learning rate and weight decay. Four configurations were tested. The resulting validation scores are summarized in Table 4.7.

The best configuration used an initial learning rate of 3×10^{-4} and a weight decay of 3×10^{-5}. This run achieved a validation accuracy of 0.84157 and a normalized edit distance of 0.87098, making it the strongest OCR candidate of the sweep. The configuration with the same learning rate but lower weight decay performed substantially worse, while the run using 5×10^{-4} degraded

sharply and converged to a much lower performance level. Increasing the learning rate further to 7×10^{-4} recovered most of the performance, but still remained below the best 3×10^{-4} run.

These results indicate that OCR fine-tuning was sensitive to the optimizer settings even within a relatively narrow search range. In particular, the stage-2 results suggest that a slightly lower learning rate than the original baseline provided more stable and ultimately better convergence on the recognition task. This behavior is visible in Figure 4.8, where the best run maintains the highest evaluation accuracy, and in Figure 4.9, where the same configuration also produces the strongest normalized edit-distance curve.

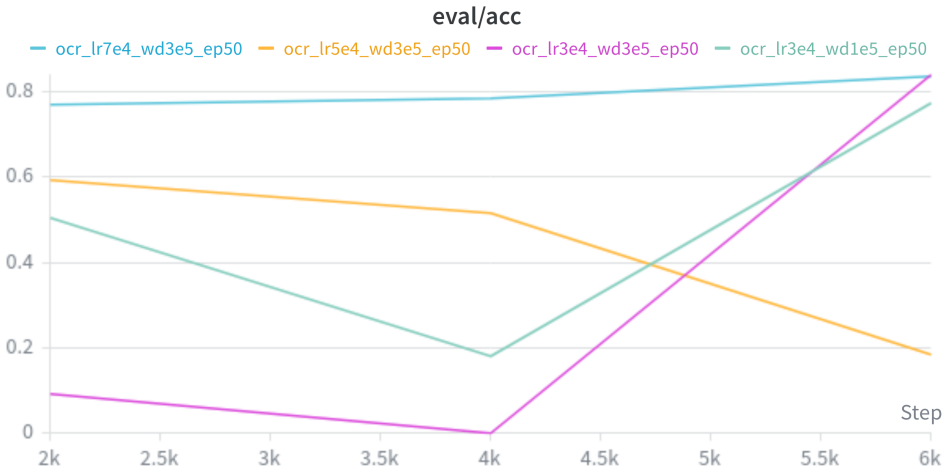


Figure 4.8: Stage 2 OCR evaluation accuracy for the learning-rate and weight-decay sweep. The configuration with learning rate 3×10^{-4} and weight decay 3×10^{-5} achieved the best validation accuracy.

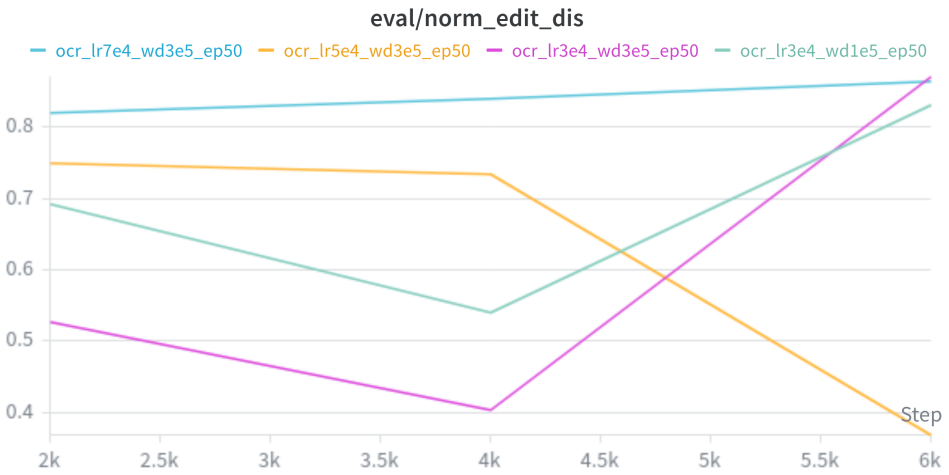


Figure 4.9: Stage 2 OCR normalized edit distance for the learning-rate and weight-decay sweep. The best-performing configuration also achieved the highest normalized edit-distance score.

Table 4.7: Stage 2 OCR hyperparameter sweep on the validation split.

Trial	Learning rate	Weight decay	Accuracy	Normalized edit distance
lr3e4_wd1e5_ep50	3×10^{-4}	1×10^{-5}	0.77531	0.83148
lr3e4_wd3e5_ep50	3×10^{-4}	3×10^{-5}	**0.84157**	**0.87098**
lr5e4_wd3e5_ep50	5×10^{-4}	3×10^{-5}	0.59603	0.74899
lr7e4_wd3e5_ep50	7×10^{-4}	3×10^{-5}	0.83653	0.86546

4.2.3 Final OCR benchmark

The final OCR benchmark compared the original fine-tuned recognizer against the best-performing stage-2 configuration on the held-out OCR test set. The results are shown in Table 4.8. The stage-2 model outperformed the initial OCR model on both recognition-quality metrics. Test accuracy improved from 0.81635 to 0.83486, while normalized edit distance improved from 0.85574 to 0.87190. In addition, the measured evaluation throughput increased slightly from 805.68 to 817.71 samples per second.

Taken together, these results show that the OCR hyperparameter tuning process produced a real improvement rather than only a validation-specific gain. Unlike the YOLO tuning experiments, where better validation performance did not translate into a better final test-set model, the OCR stage-2 winner also improved the held-out test benchmark. For this reason, the stage-2 best OCR model was selected as the final recognition candidate for later integration into the complete perception pipeline.

Table 4.8: Held-out OCR test-set comparison between the initial recognizer and the selected stage-2 OCR model.

Model	Accuracy	Normalized edit distance	Throughput (samples/s)
Initial OCR model	0.81635	0.85574	805.68
Stage-2 best OCR model	**0.83486**	**0.87190**	**817.71**

Bibliography

[1] Bai, J., Lu, F., Zhang, K., et al. (2019). Onnx: Open neural network exchange. In *Proceedings of the 2019 ACM International Conference on Multimedia Systems*, pages 460–466.

[2] Bernard, A., Hörnle, T., and Dillmann, R. (2020). Vision-based perception of control interfaces for robotic manipulation. *Robotics and Autonomous Systems*, 124:103386.

[3] Chen, T., Chen, Z., and Chen, Y. (2018). Deep learning in robotics: A review of recent research. *Advanced Robotics*, 32(5):201–214.

[4] Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., and Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3213–3223.

[5] He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2961–2969.

[6] Holland, J. et al. (2021). Service robots in the healthcare sector. *Robotics*, 10(1):47.

[7] Iguazio (2020). Nuclio: High-performance serverless event and data processing platform. `https://nuclio.io`.

[8] Jocher, G., Chaurasia, A., and Qiu, J. (2023). Yolov8: Ultralytics next-generation yolo models. *arXiv preprint arXiv:2305.09972*.

[9] Kang, Y., Kim, H., and Lee, J. (2020). Edge ai: On-device intelligence for embedded systems. *IEEE Consumer Electronics Magazine*, 9(4):20–27.

[10] Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). Microsoft coco: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 740–755.

[11] Liu, J., Fang, Y., Zhu, D., Ma, N., Pan, J., and Meng, M. Q.-H. (2021). A large-scale dataset for benchmarking elevator button segmentation and character recognition. *IEEE Transactions on Industrial Informatics*.

[12] Long, J., Shelhamer, E., and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440.

[13] Martínez, M., Montero, R., and Pérez, E. (2018). Irso: A dataset for interactive robotic service operations in indoor environments. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1234–1241.

[14] Masala, G. L. et al. (2025). Artificial intelligence and assistive robotics in healthcare: A narrative review. *International Journal of Environmental Research and Public Health*, 22(5):781.

[15] Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239).

[16] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788.

[17] Ren, S., He, K., Girshick, R., and Sun, J. (2017). Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6):1137–1149.

[18] Sekachev, B., Manovich, N., Zhiltsov, M., et al. (2020). Cvat: Computer vision annotation tool. *arXiv preprint arXiv:2009.13245*.

[19] Sun, X., Wu, J., Zhang, X., Zhang, Z., Zhang, C., Xue, T., Freeman, W., and Tenenbaum, J. (2017). Scenenet rgb-d: Can 5m synthetic images beat generic imagenet pre-training? In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2678–2687.

[20] Tan, M. and Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 6105–6114.

[21] Wang, X., Luo, C., and Zhang, Y. (2019). Vision-based perception for autonomous indoor service robots: A survey. *IEEE Transactions on Automation Science and Engineering*, 16(4):1603–1619.